

Experiment: Creating an Application in Salesforce.com using Apex Programming Language

Title

Creating an Application in Salesforce.com using Apex Programming Language

Aim

To create a cloud-based application in Salesforce.com by defining a custom object and performing CRUD (Create, Read, Update, Delete) operations using Apex Programming Language.

Problem Statement

To understand the practical implementation of cloud computing using the Salesforce platform by creating a custom application. The experiment involves creating a custom object named Student, adding required fields, writing Apex code for CRUD operations, and verifying execution through Salesforce Developer Console and application interface.

Objectives

- To understand the Salesforce cloud platform and its architecture
 - To create and manage custom objects in Salesforce
 - To understand the use of Apex Programming Language
 - To perform CRUD operations using Apex code
 - To learn server-side execution in a cloud environment
 - To understand the relationship between backend logic and user interface in Salesforce
-

Prerequisites

Before performing this experiment, the following are required:

- Basic knowledge of Cloud Computing concepts
 - Basic understanding of Object-Oriented Programming
 - Knowledge of database concepts such as objects, records, and fields
 - Salesforce Developer Account through Trailhead Playground
-

Theory

Salesforce is a cloud-based Customer Relationship Management (CRM) platform that allows users and developers to create scalable applications without managing physical infrastructure. Since Salesforce works entirely on the cloud, users can access applications from anywhere using the internet.

Salesforce provides a platform called Force.com where developers can build custom applications using tools like custom objects, workflows, reports, dashboards, and Apex programming.

Apex Programming Language

Apex is a strongly typed, object-oriented programming language developed by Salesforce. It is used for writing server-side business logic and executing database operations such as insert, update, delete, and query.

Apex is similar to Java in syntax and is mainly used when standard Salesforce features are not sufficient for business requirements.

Custom Object

A custom object is similar to a database table created by the user. Standard objects like Account, Contact, and Lead are already available in Salesforce, while custom objects are created according to project requirements.

In this experiment, a custom object named Student is created to store student-related information such as:

- Student Name
 - Age
 - Course
 - Marks
-

CRUD Operations

CRUD stands for:

- Create
- Read
- Update
- Delete

In this experiment:

- Create → Inserted a new student record
- Read → Retrieved student records using SOQL query
- Update → Updated student marks using Apex
- Delete → Deleted the student record using Apex

This experiment helps in understanding how backend programming works in a cloud platform and how data is managed using server-side logic.

Procedure

Step 1: Login to Salesforce Trailhead

First, a Salesforce Developer Account was created using Trailhead.

Website used:

<https://trailhead.salesforce.com>

After successful registration and verification, login was completed and Trailhead Playground was opened.

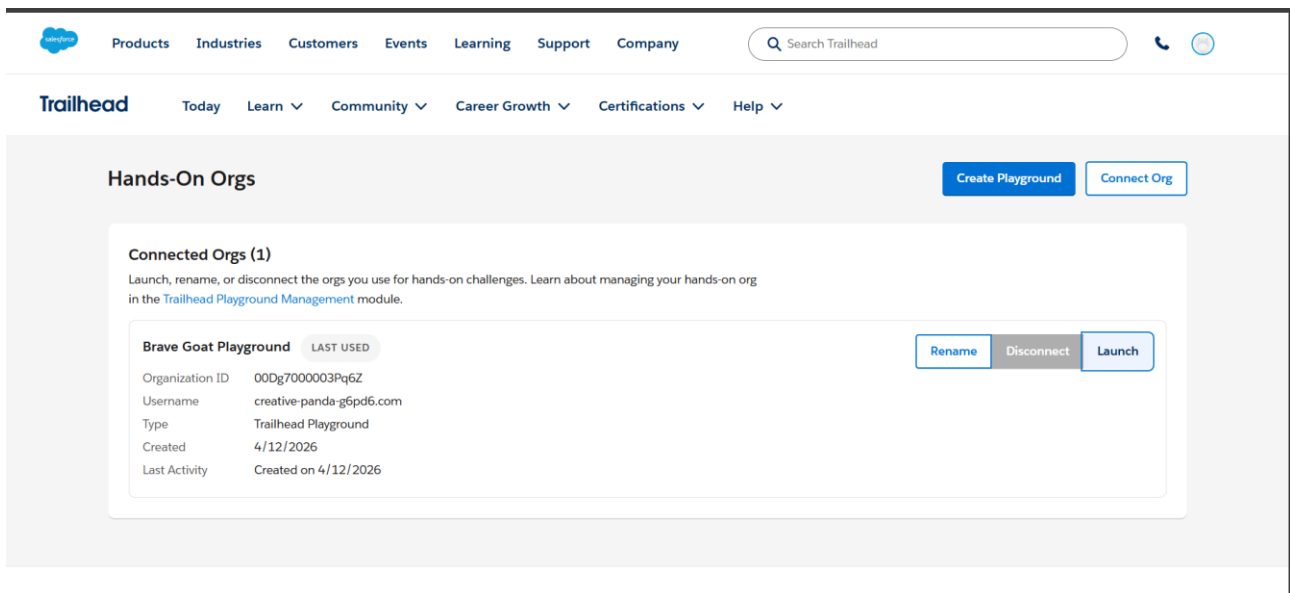


Fig 1: Salesforce Trailhead Dashboard

Step 2: Open Playground

From the profile section, the Hands-on Org (Playground) was launched. This provides a complete Salesforce development environment where practical work is performed.

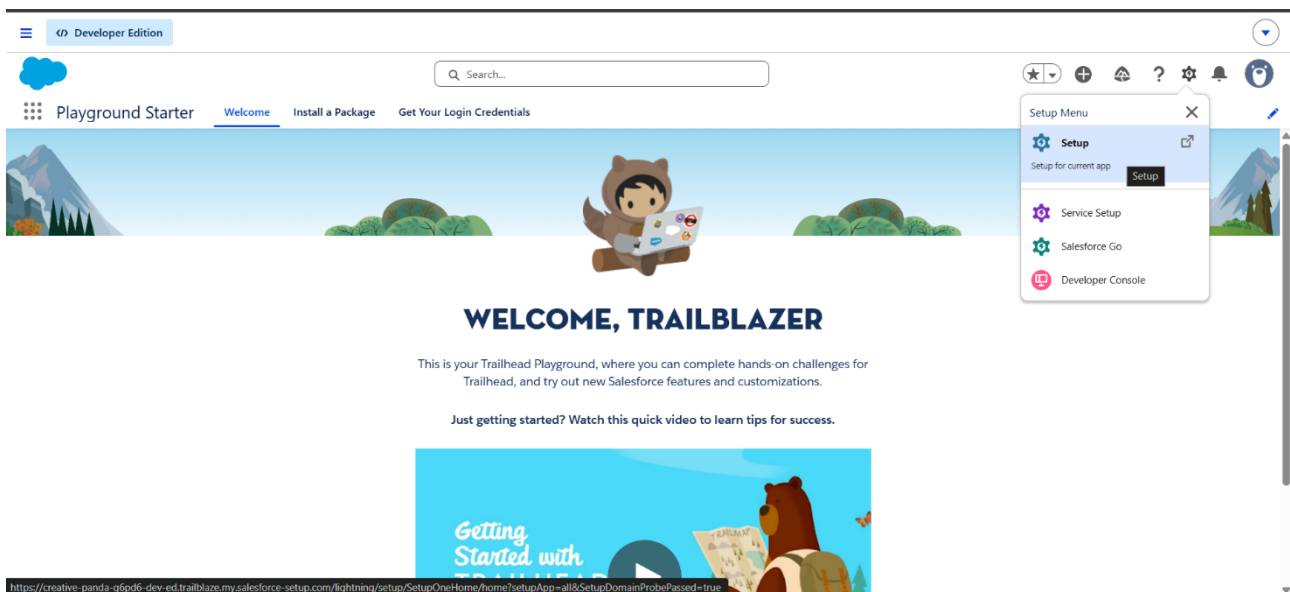


Fig 2: Playground Launch Page

Step 3: Create Custom Object

Inside Setup, Object Manager was opened and a custom object was created with the following details:

- Label: Student
- Object Name: Student
- Record Name: Student Name
- Data Type: Text

This custom object acts as a database table.

Developer Edition

Setup

Home

Object Manager ▾

Object Manager

53+ Items. Sorted by Label

[Schema Builder](#)
[Create ▾](#)

Label	API Name	Type	Description	Last Modified	Deployed
Account	Account	Standard Object			
Activity	Activity	Standard Object			
Activity History	DevopsActivityLog	Standard Object			
Alternative Payment Method	AlternativePaymentMethod	Standard Object			
API Anomaly Event Store	ApiAnomalyEventStore	Standard Object			
Approval Submission	ApprovalSubmission	Standard Object			
Approval Submission Detail	ApprovalSubmissionDetail	Standard Object			
Approval Work Item	ApprovalWorkItem	Standard Object			
Asset	Asset	Standard Object			
Asset Action	AssetAction	Standard Object			
Asset Action Event	AssetActionEvent	Standard Object			

Developer Edition

Search Setup

Star

+

🏠

?

⚙️

🔔

👤

Setup

Home

Object Manager

SETUP

New Custom Object

New Custom Object

Help for this Page

Permissions for this object are disabled for all profiles by default. You can enable object permissions in permission sets or by editing custom profiles. [Tell me more!](#) [Don't show this message again](#)

Custom Object Definition Edit

Save Save & New Cancel

Custom Object Information

Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label

Example: Account

Plural Label

Example: Accounts

Starts with vowel sound

☐

The Object Name is used when referencing the object via the API.

Object Name

Example: Account

Description

The screenshot displays the Salesforce Lightning Experience interface. At the top, the browser address bar shows the URL: `creative-panda-g6pd6-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/ObjectManager/01lg7000000mYx3/Details/view`. The page title is "Developer Edition". The main navigation bar includes "Setup", "Home", and "Object Manager". The "Object Manager" section is active, showing the "student" object details. The left sidebar lists various setup options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Restriction Rules. The main content area displays the "Details" for the "student" object, including the API Name (student__c), Singular Label (student), Plural Label (students), and various settings like Enable Reports, Track Activities, Track Field History, Deployment Status, Help Settings, and Standard Salesforce.com Help Window.

Fig 3: Creation of Student Custom Object

Step 4: Add Custom Fields

The following custom fields were created inside the Student object:

Field Name	Data Type
Age	Number
Course	Text
Marks	Number

These fields are used to store student details.

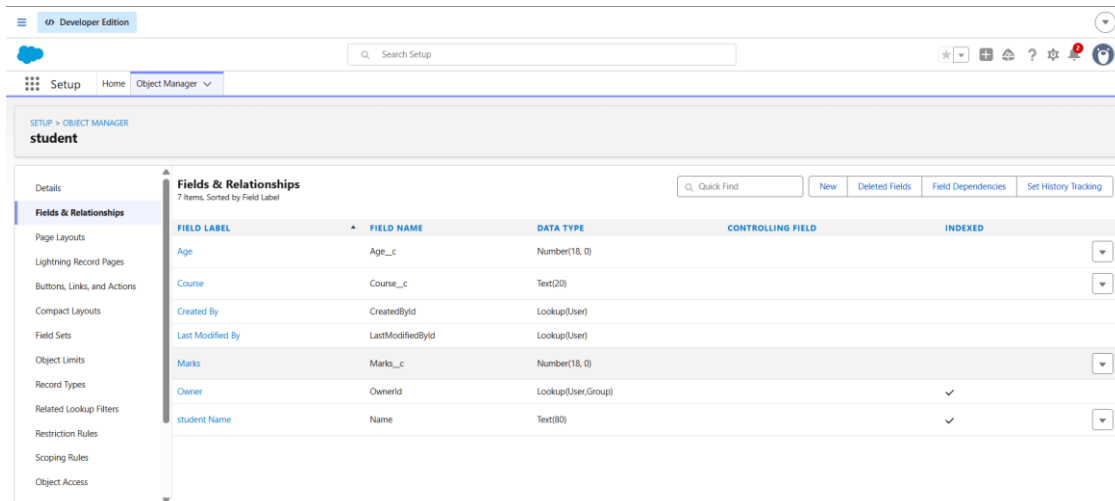


Fig 4: Creation of Custom Fields

Step 5: Open Developer Console

The Developer Console was opened from the Setup menu. It is used for writing and executing Apex code.

Navigation:

Setup → Developer Console

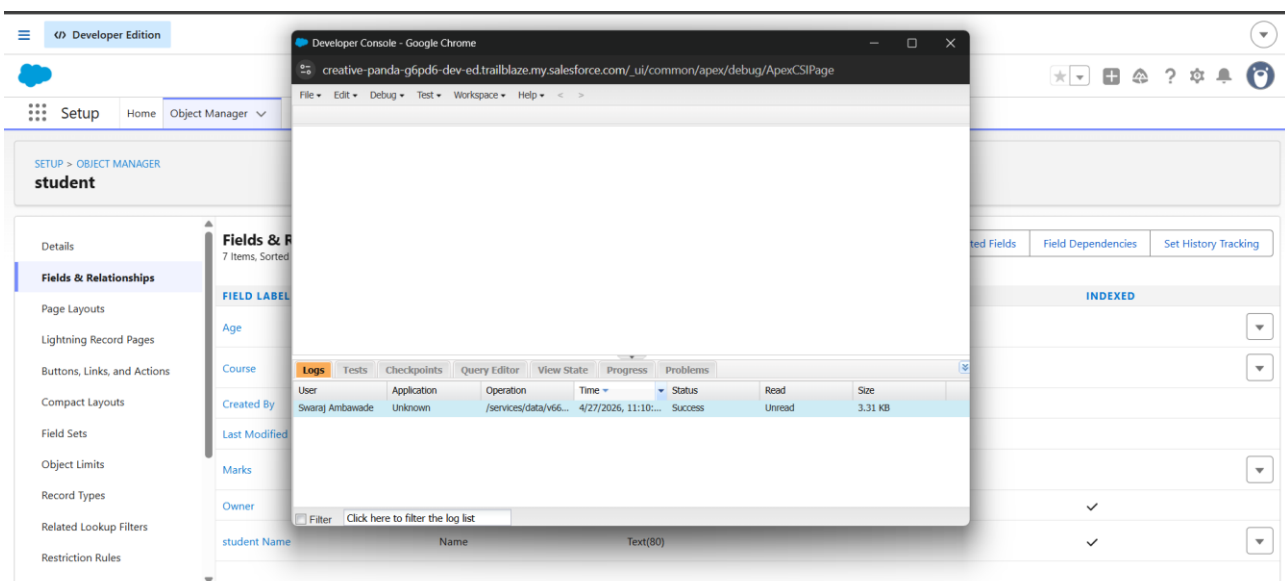


Fig 5: Developer Console Window

Step 6: CREATE Operation using Apex

The Execute Anonymous Window was opened using:

Debug → Open Execute Anonymous Window

The following Apex code was written:

```
student__c s = new student__c();
```

```
s.Name = 'Swaraj';
```

```
s.Age__c = 20;
```

```
s.Course__c = 'Computer Engineering';
```

```
s.Marks__c = 90;
```

```
insert s;
```

```
System.debug('CREATE Operation Successful');
```

This code creates and inserts a new student record.

The screenshot displays the Salesforce Developer Console interface. A window titled 'Enter Apex Code' is open, showing the following Apex code:

```
1 student__c s = new student__c();
2
3 s.Name = 'Swaraj';
4 s.Age__c = 20;
5 s.Course__c = 'Computer Engineering';
6 s.Marks__c = 90;
7
8 insert s;
9
10 System.debug('CREATE Operation Successful');
```

Below the code editor, the 'Logs' tab is selected, showing a log entry for the execution. The background shows the 'student' object manager with fields like Name, Age, Course, and Marks. Below the console, the 'student' record details are visible, showing the record created by Swaraj Ambawade on 27/04/2026 at 11:10 pm.

Field	Value
student Name	Swaraj
Age	20
Course	Computer Engineering
Marks	90
Created By	Swaraj Ambawade, 27/04/2026, 11:10 pm
Last Modified By	Swaraj Ambawade, 27/04/2026, 11:10 pm

Fig 6: CREATE Operation using Apex

Step 7: READ Operation using Apex

The following Apex code was executed to fetch student records:

```
List<student__c> students = [  
    SELECT Id, Name, Age__c, Course__c, Marks__c  
    FROM student__c  
];
```

System.debug(students);

This code retrieves all student records from the custom object.

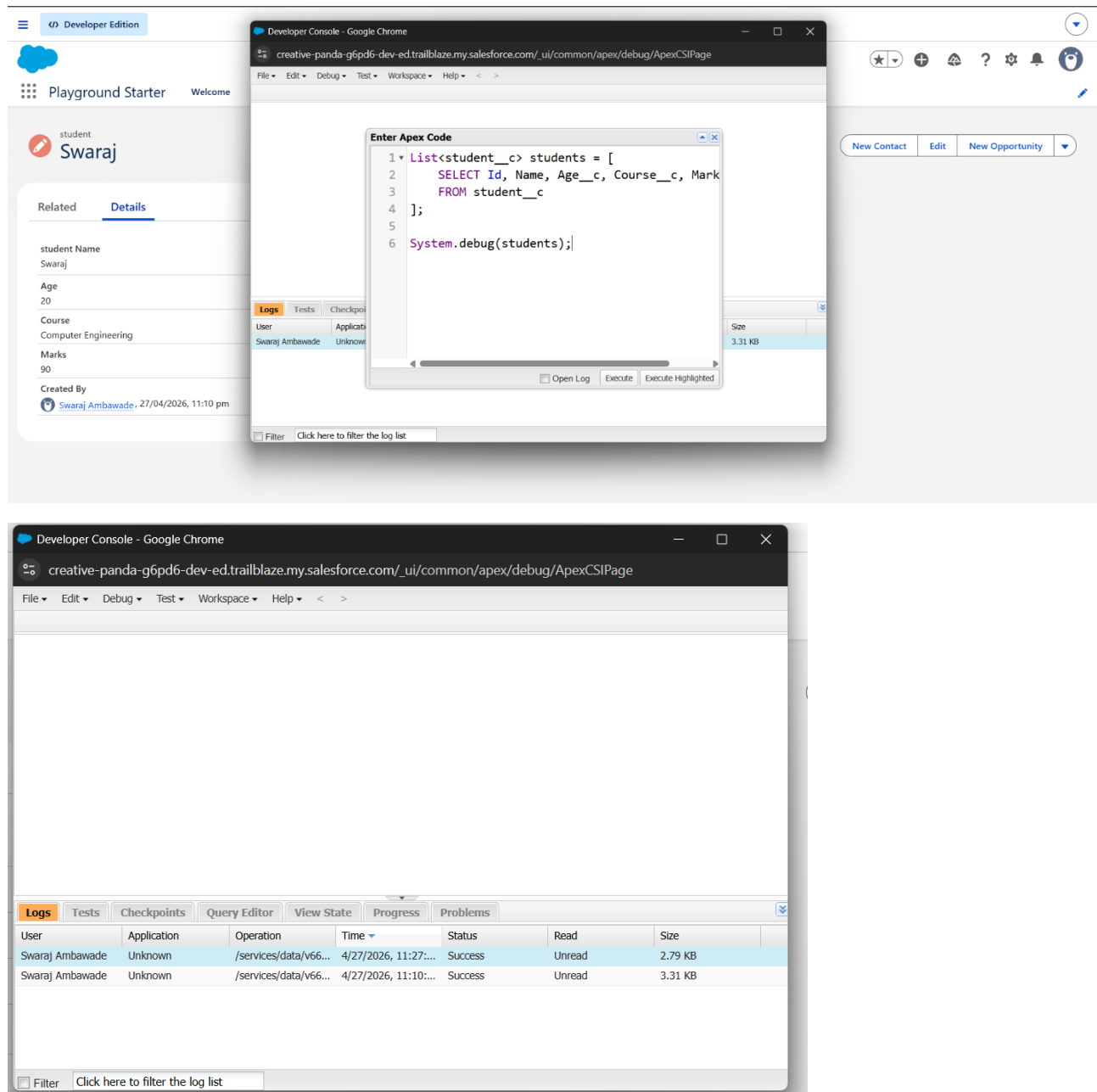


Fig 7: READ Operation using Apex

Step 8: UPDATE Operation using Apex

The following Apex code was executed to update student marks:

```
student__c s = [  
    SELECT Id, Name, Marks__c  
    FROM student__c  
    WHERE Name = 'Swaraj'  
    LIMIT 1  
];  
  
s.Marks__c = 95;  
  
update s;  
  
System.debug('UPDATE Operation Successful');
```

This updated the Marks field from 90 to 95.

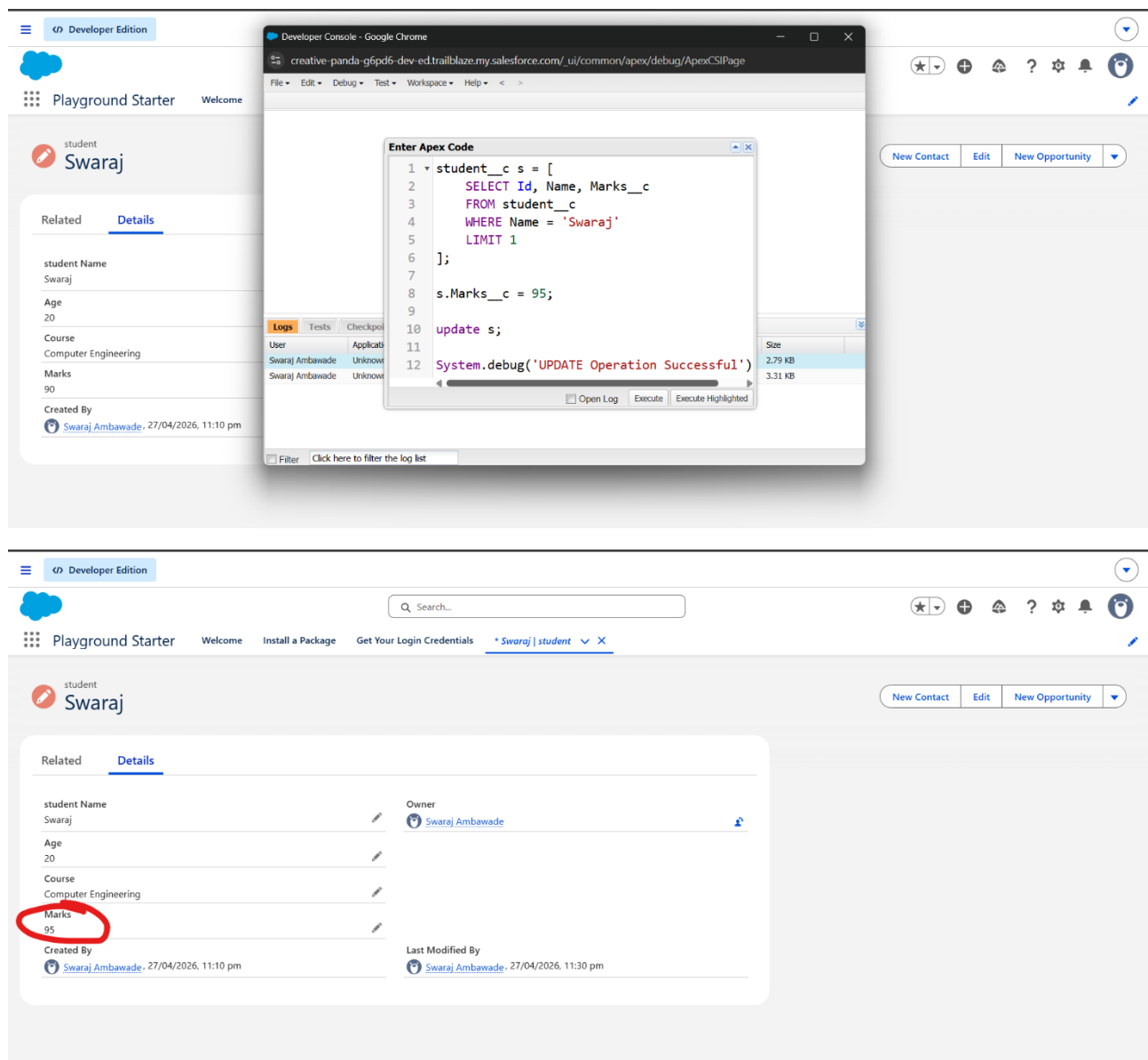


Fig 8: UPDATE Operation using Apex

Step 9: DELETE Operation using Apex

The following Apex code was executed to delete the record:

```
student__c s = [  
  SELECT Id, Name  
  FROM student__c  
  WHERE Name = 'Swaraj'  
  LIMIT 1  
];
```

```
delete s;
```

```
System.debug('DELETE Operation Successful');
```

This removed the student record from the database.

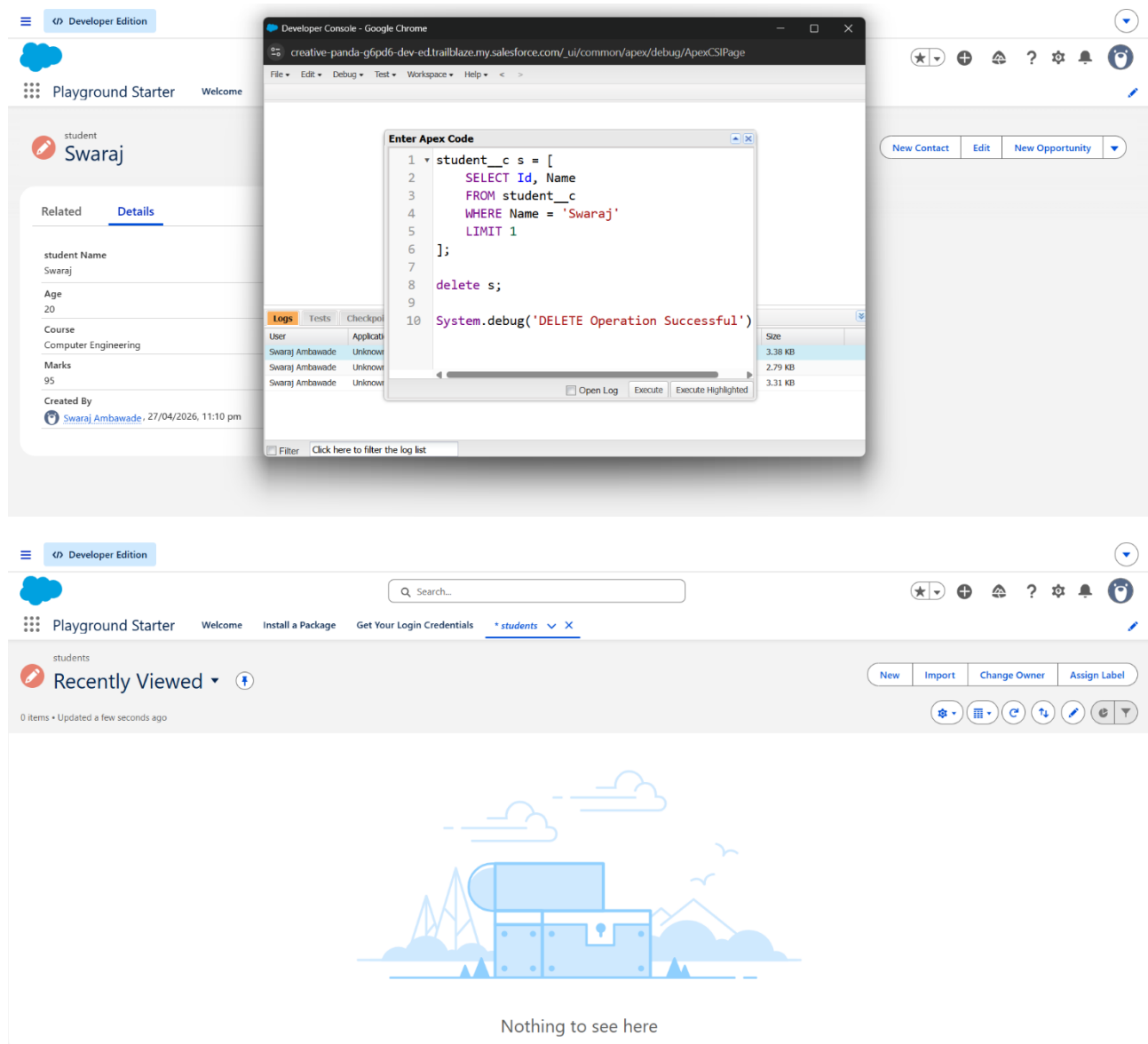


Fig 9: DELETE Operation using Apex

Step 10: Verify Debug Logs and Final Output

After each operation, Debug Logs were checked and the status showed:

Success

This confirmed successful execution of all CRUD operations.

The Student tab was opened to verify inserted and updated records before deletion.

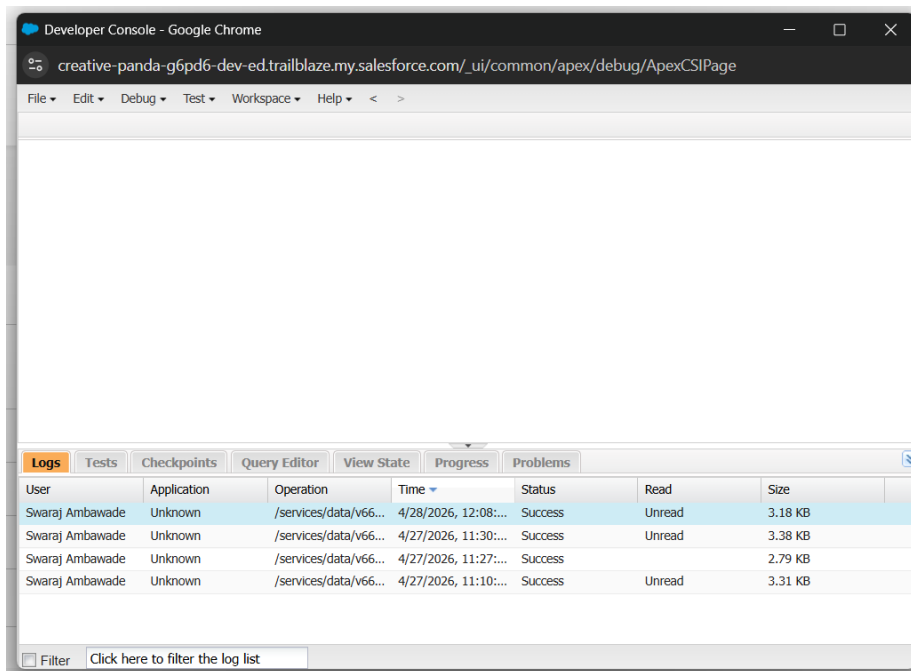


Fig 10: Debug Logs and Final Verification

Output

The Student custom object was created successfully in Salesforce.

CRUD operations were performed successfully using Apex Programming Language:

- Record inserted successfully
- Record fetched successfully
- Record updated successfully
- Record deleted successfully

All operations were verified through Debug Logs and the Salesforce user interface.

Conclusion

The experiment on creating an application in Salesforce.com using Apex Programming Language was completed successfully.

A custom object named Student was created, required fields were added, and CRUD operations were performed using Apex code.

This helped in understanding practical cloud computing implementation using Salesforce and provided knowledge of Apex programming, custom object creation, and cloud-based database operations.

The experiment also provided hands-on experience with CRM development and server-side programming in Salesforce.